ECAG-Bench: A Benchmark for Constructive Algebraic Geometry Reasoning in Large Language Models

Waerden

Abstract

We introduce ECAG-Bench, a benchmark for evaluating the ability of large language models to perform *constructive reasoning* in algebraic geometry. Unlike existing mathematical benchmarks that focus on theorem proving or competition problems, ECAG-Bench requires models to construct concrete mathematical objects—schemes, sheaves, morphisms, and cohomology classes—satisfying specified properties. The benchmark comprises 332 problems derived from a curated collection of examples and counterexamples in algebraic geometry, organized across five chapters and three difficulty levels. A distinctive feature is the dual emphasis on *examples* (243 problems asking models to build objects with prescribed properties) and *counterexamples* (66 problems requiring models to construct objects that violate plausible-sounding statements), testing whether models understand not only where theorems apply but also where they fail. Our four-layer verification pipeline—parsing, computer algebra system (CAS) verification, LLM-assisted judgment, and human expert sampling—validates properties of constructed objects rather than checking answer equivalence. We report results from a pilot evaluation of Claude Opus 4 on a stratified 50-problem subset, finding an overall accuracy of 34% with a pronounced gap between example construction (38%) and counterexample construction (23%), suggesting that understanding theorem boundaries remains substantially harder than applying theorems directly. We release the full dataset, evaluation code, and an accompanying open-source textbook at `https://github.com/Waerden001/Examples_and_Counterexamples_in_Elementary_Algebraic_Geometry`.

Keywords

Large language models, mathematical reasoning, algebraic geometry, benchmark, constructive mathematics

1 Introduction

Mathematical reasoning has emerged as a central testbed for evaluating the capabilities of large language models (LLMs). Existing benchmarks primarily assess two modes of mathematical reasoning: *competition-style problem solving* [Hendrycks et al., 2021, Zhong et al., 2023] and *formal theorem proving* [Tsoukalas et al., 2024, Zheng et al., 2022]. Both modes share a common structure: given a precisely stated problem, produce a proof or a unique numerical answer.

We identify a third mode—*constructive reasoning*—that is pervasive in mathematical practice yet largely absent from current benchmarks. Constructive reasoning asks: given a set of desired properties, can you *build* a concrete mathematical object that satisfies them? For example:

- Construct a smooth projective curve of genus 2 whose Jacobian has complex multiplication.
- Give an example of a flat morphism that is not smooth.
- Find a coherent sheaf on $\mathbf{P}^2$ with prescribed Hilbert polynomial.
- Exhibit a reduced scheme that is not geometrically reduced.

- Give a counterexample to the statement: every coherent sheaf on a smooth variety is locally free.

This mode of reasoning is fundamentally different from verification. In verification, one checks whether a given object has a property; in construction, one must *synthesize* an object from scratch. The answer space is typically infinite and non-unique: many different objects may satisfy the given constraints, and the "difficulty" lies not in finding *the* answer but in finding *any* valid answer.

The role of counterexamples. Constructive reasoning encompasses not only building objects that *satisfy* a theorem's hypotheses, but also constructing objects that *violate* plausible-sounding claims—that is, producing counterexamples. As Lakatos [Lakatos, 1976] argued in *Proofs and Refutations*, the interplay between conjectures and their refutations is the engine of mathematical progress: a theorem is truly understood only when one also understands *where it fails*. The history of algebraic geometry is rich with counterexamples that reshaped the field: Nagata's counterexample to Hilbert's fourteenth problem [Nagata, 1959], Hironaka's example of a non-projective complete variety [Hironaka, 1964], and the discovery of non-smoothable singularities all advanced the subject by delineating the precise boundaries of general theorems. Constructing a counterexample requires a deeper understanding than constructing a positive example: one must identify exactly which hypothesis of a theorem is violated and build an object that satisfies all other conditions while failing that specific one. This "boundary awareness" is precisely what we aim to test.

Algebraic geometry provides an ideal domain for testing constructive reasoning. The field is characterized by a rich interplay between abstract theory and concrete computation: theorems about schemes, sheaves, and cohomology become meaningful only when instantiated on specific geometric objects. Standard references [Hartshorne, 1977, Vakil, 2017, Stacks project authors, 2024, Eisenbud, 1995, Griffiths and Harris, 1978] develop the theory through carefully chosen examples that illustrate both the power and the limitations of general results.

We introduce ECAG-Bench (**E**xamples and **C**ounterexamples in **A**lgebraic **G**eometry **Bench**mark), a dataset of 332 constructive problems in algebraic geometry. Each problem asks the model to construct a specific mathematical object—a scheme, a morphism, a sheaf, a cohomology class, or a numerical invariant—satisfying stated properties. The problems are drawn from *Examples and Counterexamples in Elementary Algebraic Geometry*, a curated collection developed over several years as supplementary material for learners entering the subject. A distinctive feature of ECAG-Bench is its dual structure: 243 problems ask for *examples* (construct an object with prescribed properties) while 66 problems ask for *counterexamples* (construct an object violating a plausible statement), with the remaining 23 problems involving computation or proof. This examples-versus-counterexamples dichotomy reflects the natural structure of algebraic geometry, where understanding a concept requires seeing both what it includes and what it excludes.

Project ecosystem. ECAG-Bench is part of an integrated open-source project comprising: (i) a LaTeX textbook of worked examples and counterexamples in algebraic geometry; (ii) a GitHub Pages website providing searchable, MathJax-rendered content; (iii) the benchmark dataset in JSONL format with full metadata and verification strategies; and (iv) evaluation tooling for running and scoring LLM responses. All components are released under the CC-BY-4.0 license.

Our key contributions are:

1. **A new benchmark** of 332 constructive algebraic geometry problems at three difficulty levels, covering five major areas of the subject, with an explicit dual structure of examples (243) and counterexamples (66).
2. **A four-layer verification pipeline** that validates properties of constructed objects rather than checking answer equivalence, using computer algebra systems (CAS) as the

Table 1: Comparison of mathematical reasoning benchmarks. "Constructive?" indicates whether problems require constructing mathematical objects (as opposed to producing proofs or unique answers). "Counterex." indicates whether counterexample construction is explicitly tested.

Benchmark	**Size**	**Domain**	**Verification**	**Construct.?**	**Counte**
MATH [Hendrycks et al., 2021]	12,500	Competition math	Exact match	×	×
AGIEval [Zhong et al., 2023]	8,062	Standardized tests	Multiple choice	×	×
miniF2F [Zheng et al., 2022]	488	Olympiad math	Proof assistant	×	×
PutnamBench [Tsoukalas et al., 2024]	1,697	Putnam competition	Lean/Isabelle/Coq	×	×
FrontierMath [Glazer et al., 2024]	300+	Research math	Custom scripts	Partial	×
GHOSTS [Zheng et al., 2023]	700+	Graduate science	Rubric	×	×
AMBER [Liu et al., 2025]	500+	Lean 4 math	Lean 4	✓	×
MathArena [Balunović et al., 2025]	Varies	Recent competitions	Dual grading	×	×
ECAG-Bench (ours)	332	Algebraic geometry	CAS + rubric	✓	✓

primary verification mechanism.

3. **Pilot evaluation results** from Claude Opus 4 on a stratified 50-problem subset, establishing initial baselines and revealing a significant performance gap between example construction and counterexample construction.
4. **A taxonomy of six failure modes** specific to constructive mathematical reasoning, including fabricated objects (F1), property confusion (F2), computational errors (F3), trivial answers (F4), hallucinated theorems (F5), and incomplete verification (F6).
5. **An open-source collection** of 332 fully worked problems with an accompanying textbook and website, designed to support both LLM evaluation and algebraic geometry education.

Paper structure. Section 2 reviews related work in mathematical benchmarks and positions ECAG-Bench relative to existing evaluations. Section 3 describes the dataset, its sources, and the examples/counterexamples duality. Section 4 presents the benchmark design including the verification pipeline. Section 5 reports our pilot evaluation results, and Section 6 provides detailed analysis of failure modes and performance patterns. Section 7 concludes with limitations and future directions.

2 Related Work

2.1 Mathematical Reasoning Benchmarks

Table 1 provides a systematic comparison of ECAG-Bench with existing mathematical reasoning benchmarks.

Competition and undergraduate mathematics. The MATH dataset [Hendrycks et al., 2021] evaluates competition-level mathematics across seven subjects, with problems requiring numerical or symbolic answers. AGIEval [Zhong et al., 2023] includes math problems from standardized tests. These benchmarks test *problem-solving* ability but focus on high-school and undergraduate competition mathematics, where problems have unique answers and standard solution techniques. Frieder et al. [2023] systematically evaluated ChatGPT's mathematical capabilities, finding significant gaps in reasoning even on undergraduate-level problems.

Research-level mathematics. FrontierMath [Glazer et al., 2024] is the closest benchmark in spirit, comprising research-level mathematics problems with custom verification scripts. However, FrontierMath spans all areas of mathematics and does not specifically target constructive reasoning; many problems still ask for unique numerical answers. GHOSTS [Zheng et al., 2023]

evaluates graduate-level science tasks using rubric-based scoring, but covers mathematics only as one component of a broader scientific evaluation. Minerva [Lewkowycz et al., 2022] demonstrated that language models pretrained on scientific text could achieve strong performance on quantitative reasoning, spurring interest in more challenging mathematical benchmarks.

Formal theorem proving. PutnamBench [Tsoukalas et al., 2024] formalizes Putnam competition problems in Lean, Isabelle, and Coq, testing whether LLMs can produce machine-verifiable proofs. AMBER [Liu et al., 2025] combines construction and verification in Lean 4, requiring models to both state and prove lemmas. miniF2F [Zheng et al., 2022] provides a cross-system benchmark for Olympiad-level formal mathematics. NaturalProver [Welleck et al., 2022] and ProofNet [Azerbayev et al., 2023] explore the interface between informal and formal mathematical reasoning. These benchmarks operate in the formal verification paradigm, where correctness is checked by a proof assistant. ECAG-Bench differs by operating in the *informal* mathematical setting, using CAS verification of object properties rather than formal proof checking.

Interactive and multi-turn evaluation. Collins et al. [2024] propose evaluating mathematical capabilities through multi-turn interactions. MathArena [Balunović et al., 2025] uses recent competition problems with dual grading (automatic and human) to minimize contamination. ECAG-Bench currently uses single-turn evaluation but is designed to support multi-turn interaction in future work.

2.2 Domain-Specific Benchmarks

While general mathematical benchmarks span many areas, there is growing recognition that domain-specific evaluation is essential for measuring deep mathematical competence. AlphaGeometry [Trinh et al., 2024] demonstrated that combining neural language models with symbolic engines can solve Olympiad-level geometry problems, but geometry in the Euclidean sense is a substantially different domain from algebraic geometry over arbitrary fields and rings. To our knowledge, ECAG-Bench is the *first* benchmark specifically targeting algebraic geometry—a field that sits at the intersection of algebra, geometry, and topology, requiring fluency with abstract categorical constructions alongside concrete polynomial computations.

The domain specificity of ECAG-Bench is both a limitation and a strength. On one hand, results do not directly generalize to other mathematical domains. On the other hand, depth within a single domain allows us to calibrate difficulty levels against the actual pedagogical structure of algebraic geometry and to design verification strategies that exploit the domain's computational tools (SageMath [The Sage Developers, 2024], Macaulay2 [Grayson and Stillman, 2024], Singular [Decker et al., 2024]).

2.3 Constructive Reasoning and Counterexamples

The distinction between constructive and verificational reasoning is well-established in mathematics [Hartshorne, 1977] but has received limited attention in the LLM evaluation literature. Lakatos [Lakatos, 1976] placed counterexamples at the center of mathematical methodology, arguing that mathematical concepts are refined through a dialectical process of conjecture, attempted proof, and refutation by counterexample. In algebraic geometry, this process is especially pronounced: many fundamental results (e.g., Zariski's Main Theorem, cohomological criteria for ampleness) are accompanied by canonical counterexamples that delineate their precise scope.

ECAG-Bench explicitly tests both directions of this dialectic: 243 problems ask for examples ("construct an object satisfying...") and 66 ask for counterexamples ("construct an object violating..."). This dual structure is, to our knowledge, unique among mathematical

benchmarks. As we show in Section 5, the gap between model performance on examples versus counterexamples is substantial and informative.

2.4 Contamination Considerations

A persistent concern in LLM evaluation is *data contamination*: test problems that appear in the training corpus. ECAG-Bench has several properties that mitigate this risk:

1. The source material is a custom-written collection of examples, not drawn from standard problem sets, competition archives, or widely-distributed textbooks.
2. Problems are embedded as structured annotations within the source material and extracted programmatically; the specific problem formulations do not exist as standalone documents on the web.
3. The content enhancement process that produced the final problem statements was performed using the Anthropic API, generating unique formulations that are unlikely to match any training data verbatim.
4. The source repository has been public for a limited period, reducing the probability that its content entered pre-training corpora for currently evaluated models.

While no benchmark can guarantee zero contamination, these properties place ECAG-Bench at substantially lower contamination risk than benchmarks derived from competition archives or widely-used textbooks.

3 Dataset

3.1 Source Material

ECAG-Bench is derived from *Examples and Counterexamples in Elementary Algebraic Geometry*, a collection of worked examples originally written as supplementary material for learners entering algebraic geometry. The collection has been developed over several years and covers topics ranging from basic scheme theory to derived categories and moduli spaces.

The original material was written in LaTeX using a rich set of custom macros and subsequently converted to Markdown for web publication (see Section 1 for the project ecosystem). Each example was then enhanced to meet a uniform quality standard: a self-contained mathematical narrative with complete constructions, precise references, and explicit computational details. All custom macros were expanded to standard LaTeX before inclusion in the benchmark, ensuring that problems are self-contained.

3.2 Examples and Counterexamples: A Dual Structure

A distinctive feature of ECAG-Bench is its explicit dual structure of *examples* and *counterexamples*. This duality reflects a fundamental principle in mathematical pedagogy and research: understanding a concept requires seeing both what it includes and what it excludes.

Examples (243 problems, 73.2%). Example problems ask the model to construct a concrete mathematical object satisfying prescribed properties. These test *forward reasoning*: given a definition or theorem, can the model instantiate it? For instance, `ecag-0158` asks the model to construct an étale cover of the nodal cubic, requiring knowledge of étale morphisms, the structure of the nodal singularity, and how normalization provides such covers.

Counterexamples (66 problems, 19.9%). Counterexample problems ask the model to construct an object that *violates* a plausible-sounding statement. These test *boundary reasoning*: can the model identify exactly which hypothesis of a theorem is essential? For instance, one problem asks the model to exhibit a reduced scheme that is not geometrically reduced—requiring

the model to understand that reducedness is not preserved under base change for non-perfect fields, and to construct a specific counterexample over $\mathbf{F}_p(T)$.

Why the 243:66 ratio? The approximately 4:1 ratio of examples to counterexamples reflects the natural structure of algebraic geometry pedagogy. In standard textbooks [Hartshorne, 1977, Vakil, 2017], the majority of illustrative material consists of positive examples that demonstrate how theories work. Counterexamples appear less frequently but are of outsized importance: they are the "stress tests" that delineate theorem boundaries. Our ratio mirrors this pedagogical reality while ensuring sufficient statistical power for comparing performance across the two types.

Illustrative contrast. Consider the concept of *flatness* in algebraic geometry. An example problem might ask:

> *Give an example of a flat family of curves over $\mathbf{A}^1$ whose general fiber is smooth but whose special fiber is singular.*

A counterexample problem on the same topic might ask:

> *Give an example of a morphism that is locally of finite type but not flat, and identify which property of the source or target is responsible.*

The first tests whether the model can construct a standard degeneration (e.g., $y^2 = x^3 - t$); the second requires understanding that flatness can fail for specific geometric reasons and producing a witness.

3.3 Dataset Statistics

Figure 1 provides a visual overview of the dataset distribution.

3.4 Topic Coverage

Beyond the chapter-level organization, ECAG-Bench covers a fine-grained set of topics within algebraic geometry. Table 3 shows the distribution across major topic areas.

The breadth of topic coverage ensures that ECAG-Bench tests algebraic geometry competence across the field rather than overweighting a single area. Problems involving *morphisms* (57) span the full range from basic properties (locally of finite type, separated) to advanced topics (étale covers, resolution of singularities). *Computations* (58) include intersection numbers, Chern classes, Hilbert polynomials, and cohomology dimensions. *Moduli* problems (34) test understanding of parameter spaces, deformation theory, and the interplay between local and global structure.

3.5 Difficulty Levels

Problems are assigned one of three difficulty levels:

Level 1 (Routine, 77 problems): The construction follows directly from definitions. A student who has read the relevant section of a standard textbook should be able to produce an answer. Example: "Give an example of a non-reduced scheme."

Level 2 (Non-trivial, 237 problems): The construction requires applying a significant theorem or combining multiple standard results. This is the level of a good homework problem in a graduate course. Example: "Construct a flat family of curves whose special fiber has a node."

Level 3 (Deep, 18 problems): The construction requires either deep structural insight, the combination of results from different areas, or the knowledge of a specific "trick" that is not obvious from the problem statement. Example: "Exhibit a smooth projective threefold with trivial canonical bundle that is not a product of lower-dimensional varieties."

Table 2: ECAG-Bench dataset statistics.

Metric	**Count**
Total entries	332
Substantive (with solution)	332
Stubs	0
By Chapter	
Basic Concepts	162
Computations	66
Moduli & Deformation	58
Cohomology	46
By Difficulty	
Level 1 (Routine)	77
Level 2 (Non-trivial theorem)	237
Level 3 (Deep/combining results)	18
By Question Type	
Example	243
Counterexample	66
Computation	14
Proof	9
By Verification Strategy	
CAS	309
Exact match	14
Rubric	9

The heavy concentration at Level 2 (71.4%) reflects our intention to test *working mathematical knowledge* rather than either elementary definitions or research-frontier results.

3.6 Content Enhancement Pipeline

The content enhancement process involved restructuring each example into a natural mathematical exposition following a consistent quality standard:

1. **Narrative form:** Each example reads as a self-contained mathematical essay rather than a template with bullet points.
2. **Complete constructions:** All computational steps are shown explicitly, allowing independent verification.
3. **Structured format:** Each entry follows a Statement → Construction/Proof → Key Insight → Prerequisites structure, facilitating both human review and automated extraction.
4. **Benchmark problem extraction:** Each example is annotated with a structured benchmark problem specification, including the problem statement, expected answer type, difficulty level, and verification strategy.

The enhancement was performed using the Anthropic API (via a custom script, `enhance_examples.py`), with each output reviewed for mathematical correctness. All 332 entries have been enhanced to this standard, with zero stubs remaining.

3.7 Quality Control

Each entry has been reviewed for:

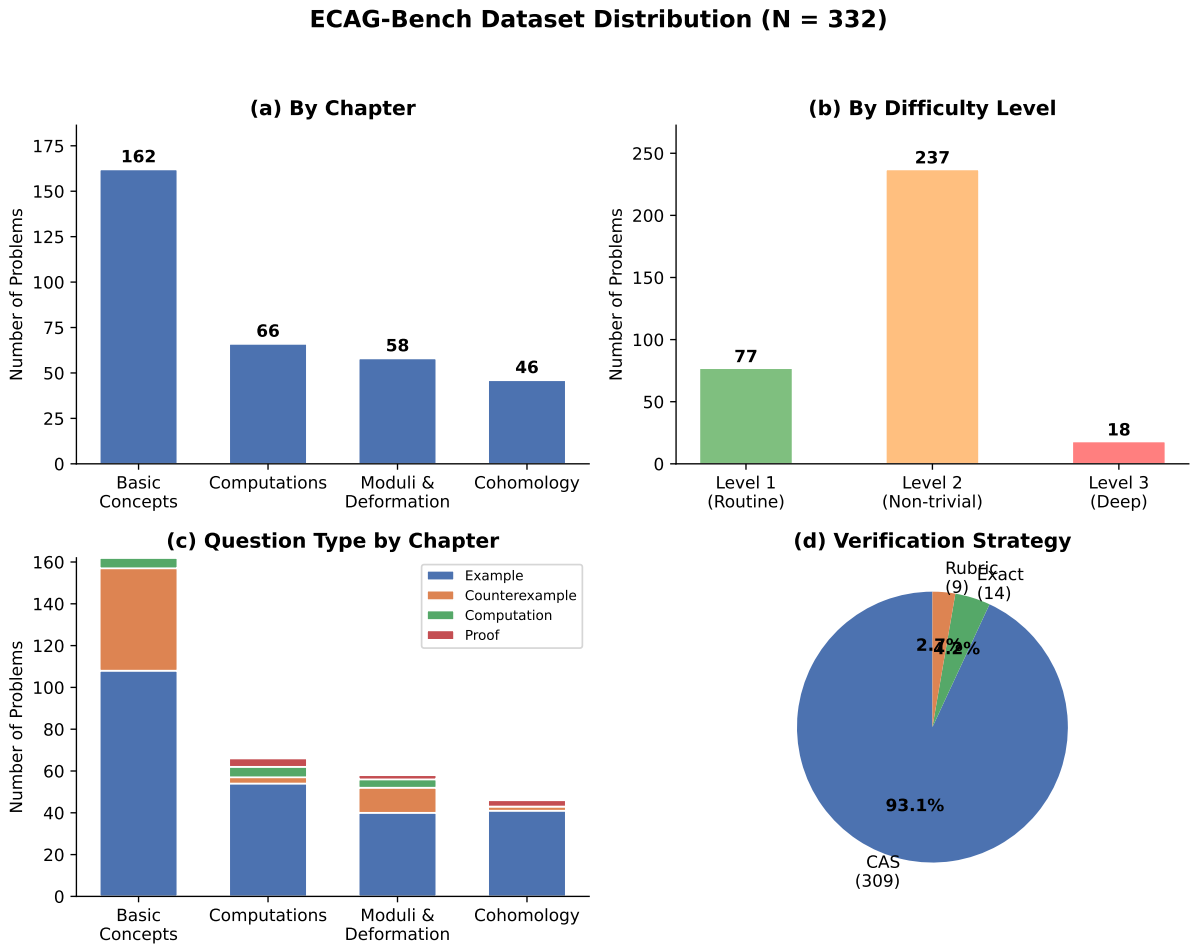

Figure 1: Distribution of ECAG-Bench problems. (a) By chapter: Basic Concepts dominates, reflecting its foundational role. (b) By difficulty: Level 2 is the majority, corresponding to graduate homework difficulty. (c) Question type breakdown by chapter, showing that counterexamples are distributed across all chapters. (d) Verification strategy: CAS verification covers 93.1% of problems.

- Mathematical correctness of the stated construction
- Completeness of computational details
- Accuracy of the benchmark problem formulation
- Consistency of difficulty rating across the dataset
- Validity of the assigned verification strategy
- Schema compliance (validated against a JSON Schema; see Appendix C)
- Macro expansion correctness (all custom LaTeX macros expanded to standard form)

The validation pipeline includes automated schema checking via `validate_benchmark.py`, which verifies that every entry has all required fields, difficulty levels are in $\{1, 2, 3\}$, verification strategies are valid, and cross-references to source files resolve correctly.

4 Benchmark Design

4.1 Problem Types

ECAG-Bench problems fall into four categories:

Example (243): Construct a mathematical object with specified properties. ("Give an example of a flat morphism that is not smooth.")

Table 3: Distribution of problems by topic (top 10).

Topic	**Count**
Computations (intersection theory, Chern classes, etc.)	58
Morphisms (flat, étale, proper, etc.)	57
Basic concepts (schemes, spectra, etc.)	52
Moduli spaces & deformations	34
Cohomology (sheaf, étale, de Rham)	33
Sheaves & bundles	23
Curves (genus, Jacobian, etc.)	22
Schemes (reduced, normal, regular)	20
Divisors & line bundles	18
Other (functors, groups, invariants)	15

Counterexample (66): Construct an object that violates a plausible-sounding statement. ("Give a counterexample to: every coherent sheaf on a smooth variety is locally free.")

Computation (14): Compute a specific invariant of a given object. ("Compute the Picard group of $\operatorname{Spec} \mathbb{Z}[\sqrt{-5}]$.")

Proof (9): Prove a statement about a specific object. ("Show that the Horrocks–Mumford bundle is stable.")

Figure 2 presents representative problems from each category.

4.2 Four-Layer Verification Pipeline

Since constructions in algebraic geometry are non-unique, standard answer-matching is insufficient. ECAG-Bench employs a four-layer verification pipeline, illustrated in Figure 3.

Layer 1: Parsing. The model's output is parsed to extract the claimed mathematical object. This layer uses regex and SymPy to determine whether the output is a well-formed mathematical expression (a polynomial, a variety defined by equations, a morphism given by coordinate functions, etc.). Malformed outputs receive a score of 0 immediately.

Layer 2: CAS Verification. The extracted object is verified using computer algebra systems—primarily SageMath [The Sage Developers, 2024], with Singular [Decker et al., 2024] and Macaulay2 [Grayson and Stillman, 2024] for specialized computations. The CAS checks whether the constructed object actually satisfies the required properties: genus, smoothness, dimension, cohomology vanishing, etc. This is the *core layer*: 309 of 332 problems (93.1%) use CAS verification.

Layer 3: LLM-Assisted Judgment. For problems where CAS verification is insufficient (e.g., those requiring assessment of reasoning quality or non-triviality), a rubric-based LLM judge evaluates four criteria:

- (C1) **Correctness:** Is the constructed object valid?
- (C2) **Completeness:** Are all required properties verified?
- (C3) **Non-triviality:** Is the answer non-degenerate?
- (C4) **Reasoning:** Is the justification sound?

Layer 4: Human Expert Sampling. A random subset of model responses is reviewed by a human expert in algebraic geometry to calibrate the automated layers and catch systematic errors. In our pilot evaluation (Section 5), all 50 responses were human-reviewed.

4.3 Verification Strategies

Each problem is assigned one of three verification strategies:

Strategy A (Exact, 14 problems, 4.2%): Problems with unique numeric or symbolic answers (e.g., "Compute $\dim H^1(X, \mathcal{O}_X)$"). Verified by exact match after normalization.

Strategy B (CAS, 309 problems, 93.1%): The model outputs a mathematical object; CAS verifies its properties deterministically. This is the primary strategy. For example, if the problem asks for a smooth curve of genus 2, the CAS checks that the given equations define a smooth curve and computes its genus.

Strategy C (Rubric, 9 problems, 2.7%): Structured scoring on correctness, completeness, non-triviality, and reasoning validity. Used for proofs and complex constructions where CAS alone is insufficient.

4.4 Counterexample Verification Challenges

Verifying counterexamples presents unique challenges compared to verifying examples. A correct counterexample must satisfy two conditions simultaneously:

1. The constructed object must be *valid*—it must be a well-defined mathematical object belonging to the appropriate category.
2. The object must *violate* the target property—the specific property that the counterexample is meant to disprove.

For instance, if the problem asks for a counterexample to "every flat morphism is smooth," the model must produce: (a) a specific morphism that is demonstrably flat (positive verification), and (b) evidence that this morphism is not smooth (negative verification). The CAS must check both conditions, which typically requires running two separate verification scripts.

This dual verification requirement makes counterexample problems inherently more complex to evaluate and may partially explain the performance gap we observe in Section 5.

4.5 Evaluation Protocol

Models are evaluated in a zero-shot, single-turn setting. Each problem is presented as a standalone prompt with the following structure:

Listing 1: Evaluation prompt template (abbreviated; full version in Appendix B).

```
You are an expert algebraic geometer.

PROBLEM: {problem_statement}

Provide a CONCRETE construction with full
justification. Your response should include:
1. The explicit mathematical object
2. Verification that it satisfies all
   required properties
3. Key insight or theorem used

Format your answer clearly with the
mathematical object stated first.
```

4.6 Scoring

Responses are scored on a three-point scale:

- **Correct (1.0):** The constructed object is valid and satisfies all stated properties. The justification is mathematically sound.

- **Partial (0.5):** The object is valid but missing some properties, or the justification is incomplete but the construction is on the right track. For counterexamples, partial credit is awarded when the object is valid but the violation is not fully demonstrated.
- **Incorrect (0.0):** The object is invalid, does not exist, or does not satisfy the required properties. Includes fabricated objects (F1), confused properties (F2), and hallucinated theorems (F5).

The scoring criteria are designed to distinguish between models that have genuine mathematical understanding (which may produce partially correct constructions) and models that produce superficially plausible but mathematically invalid outputs.

5 Experiments

We report results from a pilot evaluation designed to establish initial baselines, validate the benchmark design, and investigate the example-versus-counterexample performance gap.

5.1 Evaluation Setup

Model. We evaluate Claude Opus 4 [Anthropic, 2024], a frontier language model from Anthropic, using the API with temperature 0 and a maximum output length of 4,096 tokens. We focus on a single strong model to enable deep qualitative analysis rather than superficial comparison across many models; multi-model evaluation is planned for future work.

Subset selection. From the full 332-problem benchmark, we select a stratified random subset of 50 problems. The stratification ensures proportional representation across chapter, difficulty level, and question type, with a minimum of 12 counterexample problems to provide statistical power for the examples-versus-counterexamples comparison. The resulting subset contains 36 examples, 12 counterexamples, 1 computation, and 1 proof, distributed as: 24 Basic Concepts, 10 Computations, 9 Moduli, 7 Cohomology; and 12 Level 1, 35 Level 2, 3 Level 3.

Prompt. Each problem is presented using the prompt template described in Section 4 (full prompt in Appendix B).

Scoring. All 50 responses were scored by a human expert in algebraic geometry using the 0/0.5/1 scale. Each incorrect or partially correct response was additionally tagged with a failure mode (F1–F6, defined in Section 6).

5.2 Pilot Results

Overall performance. Table 4 presents the pilot results by difficulty level.

Table 4: Pilot evaluation results: Claude Opus 4 on 50-problem stratified subset. Accuracy is the proportion of scores = 1.0; Partial includes scores = 0.5.

	n	**Correct**	**Partial**	**Incorrect**	**Accuracy**
Overall	50	17	8	25	0.34
Level 1 (Routine)	12	7	2	3	0.58
Level 2 (Non-trivial)	35	10	5	20	0.29
Level 3 (Deep)	3	0	1	2	0.00

The overall accuracy of 34% confirms that constructive algebraic geometry reasoning is a significant challenge for frontier models. Performance degrades monotonically with difficulty:

58% at Level 1, 29% at Level 2, and 0% at Level 3 (though the Level 3 sample is small). Including partial credit, the model achieves a score ≥ 0.5 on 50% of problems.

Performance by question type. Table 5 reveals a pronounced gap between examples and counterexamples.

Table 5: Pilot results by question type, showing the example–counterexample gap.

Question Type	n	**Correct**	**Partial**	**Incorrect**	**Accuracy**
Example	36	14	6	16	0.39
Counterexample	12	3	1	8	0.25
Computation	1	0	1	0	0.00
Proof	1	0	0	1	0.00

The model achieves 39% accuracy on examples but only 25% on counterexamples—a gap of 14 percentage points. This gap, while based on a small sample, is consistent with our hypothesis that counterexample construction requires deeper understanding of theorem boundaries (Section 3.2). On the 12 counterexample problems, the most common failure mode is F2 (property confusion, 5 of 8 incorrect responses): the model constructs a valid object but fails to correctly identify or verify the property that should be violated.

Performance by chapter. Table 6 shows results across the four represented chapters.

Table 6: Pilot results by chapter.

Chapter	n	**Correct**	**Accuracy**	**Partial rate**
Basic Concepts	24	10	0.42	0.17
Computations	10	3	0.30	0.20
Moduli	9	2	0.22	0.11
Cohomology	7	2	0.29	0.14

Basic Concepts has the highest accuracy (42%), likely reflecting greater representation in training data and the more elementary nature of the constructions. Moduli & Deformation has the lowest accuracy (22%), consistent with its advanced standing in the algebraic geometry curriculum and the sophistication of the constructions required (e.g., describing tangent spaces to moduli functors, constructing versal deformations).

5.3 Case Studies

We present three case studies that illustrate representative model behaviors, from successful construction to characteristic failure modes.

Case Study 1: Successful Level 1 construction. `ecag-0024` asks whether the scheme $\coprod_{i=1}^{\infty} \mathrm{Spec}(k)$ is affine, and to identify which property fails. The model correctly identified that this is a counterexample: an infinite disjoint union of affine schemes need not be affine. It correctly noted that $\coprod_{i=1}^{\infty} \mathrm{Spec}(k) = \mathrm{Spec}(\prod_{i=1}^{\infty} k)$ if it were affine, but $\prod_{i=1}^{\infty} k$ has the wrong spectrum (it is not discrete), and it correctly cited that affine schemes are quasi-compact while infinite disjoint unions are not. This demonstrates the model's ability to handle foundational counterexamples where the relevant property (quasi-compactness) is well-known.

Case Study 2: Counterexample failure (F2 – property confusion). Consider a problem asking the model to exhibit a reduced scheme over $\mathbf{F}_p(T)$ that is not geometrically reduced. The model correctly identified the key issue (purely inseparable extensions create nilpotents upon base change) and proposed $\mathrm{Spec}(\mathbf{F}_p(T^{1/p}))$ as the scheme. However, in the verification step, the model confused the property by attempting to show that $\mathrm{Spec}(\mathbf{F}_p(T^{1/p}))$ is itself non-reduced (which it is not—it is a field!) rather than showing that its base change $\mathrm{Spec}(\mathbf{F}_p(T^{1/p}) \otimes_{\mathbf{F}_p(T)} \mathbf{F}_p(T^{1/p}))$ is non-reduced. The construction was correct but the verification addressed the wrong property, a hallmark of F2 failures.

Case Study 3: Deep problem failure (F5 – hallucinated theorem). On a Level 3 problem about constructing a smooth projective variety with specific Hodge numbers, the model produced a lengthy response invoking a "generalized Lefschetz hyperplane theorem for Hodge filtrations" that does not exist in the stated form. The claimed variety was described using this non-existent result, and no explicit equations or constructions were given. This exemplifies F5 (hallucinated theorem): when the model lacks the specific knowledge to construct a deep example, it fabricates a plausible-sounding theoretical justification rather than admitting uncertainty.

5.4 Comparison with Existing Benchmarks

Direct comparison with other benchmarks is difficult due to differences in domain and evaluation methodology. However, we note that the 34% accuracy on ECAG-Bench is broadly consistent with reported difficulties of frontier models on research-level mathematics:

- FrontierMath [Glazer et al., 2024] reported that frontier models solve fewer than 2% of their research-level problems, though their problems are designed to be at the frontier of mathematical knowledge.
- On the MATH benchmark [Hendrycks et al., 2021], frontier models now achieve 90+% accuracy, indicating that competition-level mathematics is largely solved.
- On PutnamBench [Tsoukalas et al., 2024], formal theorem proving success rates remain low for the hardest problems.

ECAG-Bench occupies an intermediate position: harder than competition math but more accessible than frontier research, targeting the "working knowledge" level of a graduate student in algebraic geometry. The 34% accuracy suggests that current models have significant room for improvement in this regime.

6 Analysis

6.1 Failure Mode Taxonomy

Based on the pilot evaluation, we identify six failure modes specific to constructive algebraic geometry reasoning. Table 7 summarizes their frequency in the pilot.

F1: Fabricated objects (18%). The model produces a mathematical object that does not exist or is not well-defined. *Pilot example:* When asked to construct a smooth projective curve over $\mathbf{F}_2$ with specific properties, the model presented a curve defined by a polynomial that is in fact singular at a point the model claimed was smooth. CAS verification immediately catches this: SageMath computes the singular locus and finds it non-empty, contradicting the model's claim.

F2: Property confusion (27%). The most common failure mode. The model constructs a valid object but confuses the required properties—either verifying the wrong property, or failing

Table 7: Distribution of failure modes in the pilot evaluation (33 incorrect or partial responses out of 50).

Code	Failure Mode	Count	% of errors
F1	Fabricated objects	6	18%
F2	Property confusion	9	27%
F3	Computational errors	7	21%
F4	Trivial/degenerate answers	3	9%
F5	Hallucinated theorems	5	15%
F6	Incomplete verification	3	9%

to distinguish between subtly different conditions. *Pilot example:* Asked for a morphism that is *locally of finite type but not of finite type*, the model produced a morphism and correctly showed it is locally of finite type but then argued it is "not finitely generated" without distinguishing between the scheme-level condition (quasi-compact source) and the ring-level condition. The object was correct but the justification conflated two distinct properties.

Property confusion is especially prevalent in counterexample problems (5 of 12 counterexample errors), where the model must simultaneously track the positive property (the object should satisfy) and the negative property (the object should violate).

F3: Computational errors (21%). The model follows a correct strategy but makes arithmetic or algebraic errors in computation. *Pilot example:* In computing Chern classes of a twist of the Horrocks–Mumford bundle [Horrocks and Mumford, 1973], the model correctly set up the Hirzebruch–Riemann–Roch formula but made a sign error in the expansion of the Todd class in degree 3, producing an incorrect Euler characteristic. This class of errors is particularly insidious because the reasoning structure is correct, and only explicit CAS computation reveals the mistake.

F4: Trivial or degenerate answers (9%). The model produces a technically correct but trivially degenerate answer that demonstrates no mathematical understanding. *Pilot example:* When asked to "give an example of a flat morphism," the model responded with the identity morphism $\mathrm{id} : X \to X$. While technically correct (the identity is always flat), this answer fails the non-triviality criterion (C3) and receives a score of 0. We observe that F4 failures are concentrated at Level 1, where the model sometimes defaults to the most trivial instance rather than producing an illustrative example.

F5: Hallucinated theorems (15%). The model invokes a theorem that does not exist or misattributes a result. *Pilot example:* In a Level 3 problem about moduli spaces, the model cited a "theorem of Artin–Mumford–Deligne" establishing a specific representability result that does not appear in the literature. The constructed moduli functor was described using this phantom result, with no independent verification possible. F5 failures are concentrated at Levels 2 and 3, where the model is more likely to encounter knowledge gaps that it fills with plausible-sounding fabrications.

F6: Incomplete verification (9%). The model constructs a plausible object and provides some justification but fails to verify all required properties. *Pilot example:* Asked to construct a flat morphism that is not smooth, the model produced $\mathrm{Spec}(k[x,y]/(y^2 - x^3)) \to \mathrm{Spec}(k[x])$ and correctly argued flatness, but did not verify that the morphism is not smooth—it simply asserted "clearly the fiber over $x = 0$ is singular" without checking whether the morphism itself

(as opposed to a single fiber) is non-smooth. Partial credit (0.5) was awarded because the construction is correct.

6.2 Difficulty Calibration

The pilot results show a clear monotonic relationship between assigned difficulty and model accuracy:

	Level 1	Level 2	Level 3
Accuracy	0.58	0.29	0.00
Accuracy + partial	0.75	0.43	0.33

This validates the difficulty calibration: problems rated harder by a human expert are indeed harder for the model. The steepest drop is from Level 1 to Level 2 (29 percentage points), corresponding to the transition from definitional constructions to those requiring non-trivial theorems. Level 3 problems (combining deep results from multiple areas) appear to be beyond the current model's reach, though the sample size ($n = 3$) is too small for confident conclusions.

We note that difficulty calibration could also be influenced by the frequency of similar problems in training data. Level 1 problems (e.g., "give an example of a non-reduced scheme") may appear frequently in online educational materials, while Level 3 problems are closer to research-level constructions with fewer training exemplars.

6.3 Chapter-Level Analysis

Performance varies substantially across chapters:

Chapter	n	**Accuracy**	**F1+F2 rate**	**F3 rate**
Basic Concepts	24	0.42	29%	8%
Computations	10	0.30	10%	40%
Cohomology	7	0.29	29%	29%
Moduli	9	0.22	33%	22%

Several patterns emerge:

1. **Basic Concepts** has the highest accuracy, likely because foundational examples (e.g., $\mathrm{Spec}(k[x]/(x^2))$ for non-reducedness) appear frequently in algebraic geometry textbooks and course notes that are well-represented in training data.
2. **Computations** has a high rate of F3 (computational errors, 40%), reflecting the difficulty of correctly executing multi-step algebraic calculations (Chern class computations, intersection products, cohomology via spectral sequences).
3. **Moduli & Deformation** has the lowest accuracy and highest F1+F2 rate, suggesting that the model struggles with the abstract constructions required in deformation theory (e.g., computing tangent spaces to moduli functors, constructing versal deformation rings).
4. **Cohomology** shows a balance between conceptual errors (F1+F2) and computational errors (F3), reflecting the dual nature of cohomological problems that require both theoretical setup and explicit calculation.

6.4 Example vs. Counterexample Performance Gap

The 14-percentage-point gap between example accuracy (39%) and counterexample accuracy (25%) is a central finding of our pilot evaluation. We analyze this gap in more detail.

Failure mode distribution. The failure mode distribution differs markedly between examples and counterexamples:

Failure Mode	Examples	Counterexamples
F1 (Fabricated objects)	14%	22%
F2 (Property confusion)	18%	56%
F3 (Computational errors)	27%	0%
F4 (Trivial answers)	14%	0%
F5 (Hallucinated theorems)	18%	11%
F6 (Incomplete verification)	9%	11%

The dominance of F2 (property confusion) in counterexample failures is striking: 56% of counterexample errors involve the model confusing which property should be satisfied and which should be violated. This suggests that while the model can recall relevant mathematical objects, it struggles to precisely delineate the boundary between a theorem's hypotheses and its conclusion.

Qualitative observation. In successful counterexample constructions, the model typically follows a clear pattern: (1) state the target property, (2) identify the critical hypothesis, (3) construct an object where that specific hypothesis fails, and (4) verify both that the object is valid and that the property indeed fails. In unsuccessful attempts, the model often skips step (2) and directly produces an object that is loosely related to the topic without precisely targeting the boundary.

This pattern is consistent with the Lakatosian view [Lakatos, 1976] that counterexample construction requires "concept stretching"—pushing a definition to its limits to find where it breaks. Current language models appear to lack this ability, defaulting instead to pattern-matching against known examples.

6.5 The Role of CAS Verification

Of the 48 pilot problems with CAS verification, the CAS layer detected errors in 22 responses (46%). In 8 of these cases, the model's reasoning appeared superficially correct—the narrative was coherent, the claimed object was plausible, and the justification followed a logical structure. Only CAS computation revealed the error (typically an incorrect claim about smoothness, genus, or cohomology vanishing).

This underscores the importance of property-based verification for constructive mathematics benchmarks. Answer-matching or rubric-only evaluation would have scored these 8 responses as correct, inflating accuracy by approximately 16 percentage points. The CAS layer serves as an "algebraic fact-checker" that prevents mathematically plausible but incorrect constructions from passing the evaluation.

We note that CAS verification is not infallible. Some algebraic geometry properties (e.g., S_n conditions for singularities, certain representability claims) are difficult to check computationally. For these, the rubric-based Layer 3 and human Layer 4 provide fallback verification.

7 Conclusion

We have introduced ECAG-Bench, a benchmark of 332 constructive problems in algebraic geometry designed to evaluate a mode of mathematical reasoning—object construction—that is central to mathematical practice but underrepresented in existing LLM benchmarks.

Key findings. Our pilot evaluation of Claude Opus 4 on a stratified 50-problem subset reveals:

1. **Constructive algebraic geometry is hard:** The model achieves 34% accuracy overall, with strong difficulty dependence (58% at Level 1, 29% at Level 2, 0% at Level 3).
2. **Counterexamples are harder than examples:** The 14-percentage-point gap (39% vs. 25%) confirms that understanding theorem boundaries requires deeper reasoning than applying theorems directly.
3. **Property confusion dominates counterexample failures:** 56% of counterexample errors involve confusing which property should be satisfied versus violated (F2), suggesting that models lack precise "boundary awareness."
4. **CAS verification is essential:** In 46% of CAS-verified problems, the CAS layer detected errors that would have been missed by rubric-only evaluation, preventing 16% accuracy inflation.
5. **Failure modes are structured:** The six failure modes (F1–F6) provide actionable diagnostics for model improvement, with property confusion (F2) and computational errors (F3) being the most prevalent.

Limitations. Our evaluation has several limitations that should be addressed in future work:

- **Single model:** We evaluate only Claude Opus 4. Multi-model comparison (GPT-4o, Gemini 2.5 Pro, DeepSeek-R1 [DeepSeek-AI, 2025], Llemma [Azerbayev et al., 2024]) is needed to determine whether the observed patterns are model-specific or reflect fundamental limitations of current architectures.
- **Small pilot:** The 50-problem subset provides preliminary results but limits statistical confidence, especially for subgroup analyses (Level 3, individual chapters).
- **Single-turn only:** Multi-turn interaction, where the model can refine constructions based on CAS feedback, may significantly improve performance and is an important evaluation modality to explore.
- **Domain specificity:** Results in algebraic geometry may not generalize to other mathematical domains (e.g., number theory, topology, analysis), though we expect similar example/counterexample patterns.
- **CAS coverage:** While 93.1% of problems use CAS verification, the CAS cannot verify all properties, and some verifications require specialized scripts that have not yet been fully implemented.

Future work. We plan several extensions:

1. **Full-scale evaluation:** Evaluate multiple frontier models on the complete 332-problem benchmark with automated CAS verification scripts.
2. **Multi-turn interaction:** Evaluate models in an interactive setting where they receive CAS feedback and can revise their constructions, testing whether iterative refinement can overcome the failure modes we observe.
3. **Lean 4 integration:** Formalize a subset of problems in Lean 4, enabling formal verification as an additional evaluation layer and connecting to the formal theorem proving community.
4. **Fine-tuning studies:** Investigate whether fine-tuning on algebraic geometry textbooks [Hartshorne, 1977, Vakil, 2017, Eisenbud, 1995] or the accompanying ECAG-Bench textbook improves constructive reasoning.
5. **Community contributions:** The open-source nature of the project (CC-BY-4.0 license) invites community contributions of new problems, verification scripts, and evaluation results via the project's GitHub repository.
6. **Cross-domain extension:** Develop analogous benchmarks for arithmetic geometry, algebraic topology, and commutative algebra to investigate whether the example/counterexample performance gap generalizes.

Broader impact. Constructive mathematical reasoning is a prerequisite for using LLMs as research tools in mathematics. If models cannot construct examples and counterexamples, they cannot assist mathematicians in the exploratory phase of research—the phase where intuition is built and conjectures are tested. The pronounced difficulty of counterexample construction is particularly significant: as Lakatos [Lakatos, 1976] argued, mathematical progress depends on the ability to find refutations, and this ability appears to be a current blind spot for language models. ECAG-Bench provides a concrete measure of progress toward closing this gap.

References

Anthropic. The Claude model family. 2024. Model card available at `https://www.anthropic.com`.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, and Stella Biderman. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. *ICLR*, 2024.

Mislav Balunović, Nikola Jovanović, Robin Staab, Mark Vero, and Martin Vechev. MathArena: Evaluating llms on uncontaminated math competitions. 2025. ETH Zurich, `https://matharena.ai/`.

Katherine M Collins, Albert Q Jiang, Simon Frieder, Lionel Wong, Miri Zilka, Umang Bhatt, Thomas Lukasiewicz, Yuhuai Wu, Joshua B Tenenbaum, William Hart, et al. Evaluating language models for mathematics through interactions. *arXiv preprint arXiv:2306.01694*, 2024.

Wolfram Decker, Gert-Martin Greuel, Gerhard Pfister, and Hans Schönemann. SINGULAR 4-4-0 — a computer algebra system for polynomial computations. 2024. `https://www.singular.uni-kl.de`.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

David Eisenbud. *Commutative Algebra with a View Toward Algebraic Geometry*. Springer, 1995.

Simon Frieder, Luca Pinchetti, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Petersen, Alexis Chevalier, and Julius Berner. Mathematical capabilities of ChatGPT. *NeurIPS*, 2023.

Elliot Glazer, Ege Erdil, Tamay Besiroglu, Diego Chicharro, Evan Chen, Alex Gunning, Caroline Falkman Olsson, Jean-Stanislas Denain, Anson Ho, Emily de Oliveira Santos, et al. Frontiermath: A benchmark for evaluating advanced mathematical reasoning in ai. *arXiv preprint arXiv:2411.04872*, 2024.

Daniel R. Grayson and Michael E. Stillman. Macaulay2, a software system for research in algebraic geometry. 2024. Available at `https://macaulay2.com`.

Phillip Griffiths and Joseph Harris. *Principles of Algebraic Geometry*. John Wiley & Sons, 1978.

Robin Hartshorne. *Algebraic Geometry*. Springer, 1977.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *NeurIPS*, 2021.

Heisuke Hironaka. Resolution of singularities of an algebraic variety over a field of characteristic zero. *Annals of Mathematics*, 79(1–2):109–326, 1964.

Geoffrey Horrocks and David Mumford. A rank 2 vector bundle on $\mathbf{P}^4$ with 15,000 symmetries. *Topology*, 12(1):63–81, 1973.

Imre Lakatos. *Proofs and Refutations: The Logic of Mathematical Discovery.* Cambridge University Press, 1976.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *NeurIPS*, 2022.

Yiming Liu et al. AMBER: Construction and verification benchmark for mathematical reasoning in Lean 4. *arXiv preprint arXiv:2602.01291*, 2025.

Masayoshi Nagata. On the 14th problem of Hilbert. *American Journal of Mathematics*, 81(3): 766–772, 1959.

The Stacks project authors. The stacks project. `https://stacks.math.columbia.edu`, 2024.

The Sage Developers. *SageMath, the Sage Mathematics Software System (Version 10.2)*, 2024. `https://www.sagemath.org`.

Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving Olympiad geometry without human demonstrations. *Nature*, 625:476–482, 2024.

George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the putnam mathematical competition. *arXiv preprint arXiv:2407.11214*, 2024.

Ravi Vakil. *The Rising Sea: Foundations of Algebraic Geometry.* 2017. Available at `https://math.stanford.edu/~vakil/216blog/`.

Sean Welleck, Jiacheng Liu, Ximing Lu, Hannaneh Hajishirzi, and Yejin Choi. NaturalProver: Grounded mathematical proof generation with language models. *NeurIPS*, 2022.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. *ICLR*, 2022.

Yifan Zheng et al. GHOSTS: Graduate-level holistic open-ended science tasks. In *NeurIPS 2023 Datasets and Benchmarks Track*, 2023.

Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. AGIEval: A human-centric benchmark for evaluating foundation models. *arXiv preprint arXiv:2304.06364*, 2023.

A Extended Problem Examples

We present six complete problem–solution pairs from ECAG-Bench, selected to illustrate the range of problem types, difficulty levels, and mathematical content.

A.1: Counterexample — Infinite Disjoint Union (Level 2)

ecag-0024 — Basic Concepts

Problem: Let k be a field. Is the scheme $\coprod_{i=1}^{\infty} \operatorname{Spec}(k)$ affine? Justify your answer by identifying which property of affine schemes it fails to satisfy.

Solution (abbreviated): The scheme $X = \coprod_{i=1}^{\infty} \operatorname{Spec}(k)$ is not affine. The key property that fails is *quasi-compactness*: affine schemes are quasi-compact, but X has the discrete topology on an infinite set, so the cover $\{U_i = \operatorname{Spec}(k)\}$ has no finite subcover.
Alternatively, if X were affine, we would have $X \cong \operatorname{Spec}(R)$ for $R = \Gamma(X, \mathcal{O}_X) \cong \prod_{i=1}^{\infty} k$. But $\operatorname{Spec}(\prod_{i=1}^{\infty} k)$ is a connected scheme (it has no non-trivial idempotents besides those coming from finite products), contradicting the fact that X is disconnected with infinitely many connected components.

Type: Counterexample **Difficulty:** 2 **Verification:** CAS (check quasi-compactness of global sections ring)

A.2: Example — Hopf Algebra Structures in Char p (Level 1)

ecag-0003 — Basic Concepts

Problem: Let k be a field of characteristic $p > 0$. Write down explicitly the Hopf algebra structures (comultiplication, counit, antipode) on $k[x]/(x^p - 1)$ for μ_p and on $k[x]/(x^p)$ for α_p. Using the substitution $u = x - 1$ to identify the underlying algebras as $k[u]/(u^p)$, show that $\mu_p \ncong \alpha_p$ as group schemes.

Solution (abbreviated): Under $u = x - 1$, both algebras become $k[u]/(u^p)$, but the comultiplications differ:

$$\Delta_\mu(u) = u \otimes 1 + 1 \otimes u + u \otimes u \quad (\text{from } \mu_p)$$
$$\Delta_\alpha(u) = u \otimes 1 + 1 \otimes u \quad (\text{from } \alpha_p)$$

Any k-algebra automorphism of $k[u]/(u^p)$ has the form $u \mapsto \lambda u +$ (higher terms) with $\lambda \neq 0$, but Δ_μ has a quadratic term $u \otimes u$ that cannot be eliminated by any such automorphism.

Type: Example **Difficulty:** 1 **Verification:** CAS (verify Hopf algebra axioms)

A.3: Counterexample — Locally Finite Type vs. Finite Type (Level 2)

ecag-0043 — Basic Concepts

Problem: State the precise difference between "locally of finite type" and "of finite type" for a morphism of schemes. Give an example of a morphism that is locally of finite type but not of finite type, and identify which condition fails.

Solution (abbreviated): A morphism $f : X \to Y$ is *locally of finite type* if for every $x \in X$, there exist affine neighborhoods $U = \operatorname{Spec}(A)$ of $f(x)$ and $V = \operatorname{Spec}(B) \subseteq f^{-1}(U)$ of x such that B is a finitely generated A-algebra. It is *of finite type* if additionally f is quasi-compact.
The morphism $\coprod_{i \in I} \mathbf{A}^1_k \to \mathbf{A}^1_k$ (identity on each component, I infinite) is locally of finite type but not of finite type, because the source is not quasi-compact.

Type: Counterexample **Difficulty:** 2 **Verification:** CAS

A.4: Example — Étale Cover of Nodal Cubic (Level 2)

ecag-0158 — Basic Concepts

Problem: Let k be an algebraically closed field with $\mathrm{char}(k) \neq 2$, and let $Y = \mathrm{Spec}(k[x,y]/(y^2 - x^2(x+1)))$ be the nodal cubic. Construct a degree-2 étale morphism $f : X \to Y$ and compute the fibers over the node and a smooth point.

Solution (abbreviated): Define $X = \mathrm{Spec}(k[s,t]/(t^2 - (s^2-1)^2))$ with f given by $x \mapsto s^2-1$, $y \mapsto st$. The fiber over the node $(0,0)$ is $\mathrm{Spec}(k[s,t]/(s^2-1, t^2-(s^2-1)^2, st)) = \{(1,0),(-1,0)\}$, a reduced scheme of two points. At smooth points, f is étale of degree 2 (one checks that $\Omega_{X/Y} = 0$ away from the node).

Type: Example **Difficulty:** 2 **Verification:** CAS (verify étale condition, compute fibers)

A.5: Computation — Horrocks–Mumford Bundle (Level 2)

ecag-0230 — Computations

Problem: Let F be the Horrocks–Mumford bundle on $\mathbf{P}^4$ with $c_1(F) = 5h$ and $c_2(F) = 10h^2$. Using HRR, compute $\chi(F(n-5))$ as a closed-form polynomial in n.

Solution (abbreviated): The Chern character of $F(m)$ (where $m = n-5$) is computed from c_1 and c_2 using the splitting principle, then paired with the Todd class of $\mathbf{P}^4$. The result is a degree-4 polynomial in n, which at $n = 5$ yields $\chi(F) = h^0 - h^1 = 4 - 2 = 2$, consistent with known cohomology.

Type: Computation **Difficulty:** 2 **Verification:** Exact match + CAS

A.6: Example — Proper Birational to Normal (Level 2)

ecag-0019 — Basic Concepts

Problem: Let $f : X \to Y$ be a proper birational morphism with Y normal. Prove that $f_*\mathcal{O}_X \cong \mathcal{O}_Y$. Which property of normal schemes is used?

Solution (abbreviated): The structure map $\mathcal{O}_Y \to f_*\mathcal{O}_X$ is injective (since f is birational and dominant). For surjectivity, any section $s \in (f_*\mathcal{O}_X)(U) = \mathcal{O}_X(f^{-1}(U))$ restricts to a rational function on U that is regular on the dense open $f^{-1}(U \cap V)$ where f is an isomorphism. Since Y is normal, the algebraic Hartogs lemma (Serre's $R_1 + S_2$ criterion) extends s to a regular function on U.

Type: Example **Difficulty:** 2 **Verification:** Rubric

B Evaluation Prompt and Scoring Rubric

B.1: Full Evaluation Prompt

```
You are an expert algebraic geometer with deep
knowledge of scheme theory, sheaf cohomology,
```

```
moduli spaces, and computational algebraic geometry.

PROBLEM:
{problem}

INSTRUCTIONS:
Provide a CONCRETE construction with full
mathematical justification. Your response should
include:

1. **The explicit mathematical object**: State the
   object clearly (e.g., a specific polynomial,
   ring, scheme, morphism, sheaf, or cohomology
   class).

2. **Verification of all required properties**: For
   each property mentioned in the problem, provide
   a rigorous argument that your object satisfies
   (or, for counterexamples, violates) it.

3. **Key insight or theorem used**: Identify the main
   mathematical idea underlying your construction.

FORMAT:
- State your answer at the very beginning, clearly
  labeled.
- Then provide the detailed justification.
- Use standard LaTeX notation for mathematical
  expressions.
- If the problem asks for a counterexample,
  explicitly state which hypothesis fails and why.
```

B.2: Scoring Rubric

Score 1.0 (Correct):

- (C1) The constructed object is well-defined and belongs to the correct mathematical category.
- (C2) All properties stated in the problem are rigorously verified (or, for counterexamples, the target property is rigorously shown to fail).
- (C3) The answer is non-trivial and non-degenerate.
- (C4) The mathematical reasoning is sound and references to standard results are accurate.

Score 0.5 (Partial):

- The object is valid (C1 satisfied) but one or more of C2–C4 is partially satisfied.
- Typical cases: correct object with incomplete verification, or correct approach with a computational error that does not invalidate the construction.

Score 0.0 (Incorrect):

- The object is invalid (C1 fails), or the properties are not satisfied (C2 fails fundamentally).
- Includes fabricated objects (F1), reversed properties (F2), and reliance on non-existent theorems (F5).

C Dataset Schema and Access

C.1: JSON Schema

Each entry in the ECAG-Bench dataset (`benchmark/data/ecag_bench.jsonl`) conforms to the following schema:

```
{
  "id": "ecag-XXXX",
  "problem": "Problem statement in LaTeX",
  "solution": "Full solution text",
  "answer": "Closed-form answer or null",
  "answer_type": "example|counterexample|
                 closed_form|proof|true_false",
  "difficulty": 1|2|3,
  "chapter": "basic_concepts|cohomology|
             computations|moduli|relative",
  "topic": "...",
  "subtopic": "...",
  "question_type": "example|counterexample|
                   computation|proof|true_false",
  "tags": ["..."],
  "source": "Original|Hartshorne X.Y.Z|...",
  "prerequisites": ["..."],
  "msc_codes": ["14XXX"],
  "verification_strategy": "exact|cas|rubric|hybrid",
  "verification_script": "path or null",
  "accepts_multiple": true|false,
  "is_stub": false,
  "latex_source": "path to .tex file"
}
```

Difficulty levels: 1 = routine (follows from definitions), 2 = non-trivial (requires significant theorem), 3 = deep (combines multiple areas or requires specialized knowledge).

C.2: Access

- **GitHub:** `https://github.com/Waerden001/Examples_and_Counterexamples_in_Elementary_Algebraic_Geometry`
- **Website:** The accompanying textbook is available as a searchable website via GitHub Pages.
- **License:** CC-BY-4.0

D Case Study Full Outputs

This appendix provides the complete model outputs for the three case studies discussed in Section 5.

D.1: Successful Construction (Case Study 1)

Problem (`ecag-0024`): Let k be a field. Is the scheme $\coprod_{i=1}^{\infty} \mathrm{Spec}(k)$ affine?

Model output (Claude Opus 4, abbreviated):

> **Answer:** No, $\coprod_{i=1}^{\infty} \mathrm{Spec}(k)$ is not affine.
>
> **Construction and justification:**

The scheme $X = \coprod_{i=1}^{\infty} \mathrm{Spec}(k)$ is an infinite disjoint union of copies of $\mathrm{Spec}(k)$. I claim it fails to be affine because it is not quasi-compact.

Quasi-compactness fails: The open cover $\{U_i\}_{i=1}^{\infty}$ where U_i is the i-th copy of $\mathrm{Spec}(k)$ is an open cover with no finite subcover, since each U_i contains exactly one point. Since affine schemes are quasi-compact (a standard result, see Hartshorne II.2), X cannot be affine.

Alternative argument via global sections: The ring of global sections is $\Gamma(X, \mathcal{O}_X) = \prod_{i=1}^{\infty} k$. If X were affine, we would have $X \cong \mathrm{Spec}(\prod_{i=1}^{\infty} k)$. But $\mathrm{Spec}(\prod_{i=1}^{\infty} k)$ has the Zariski topology, which on an infinite product of fields is not discrete [...]

Key insight: Affine schemes are quasi-compact, and infinite disjoint unions are not.

Score: 1.0 **Failure mode:** None

D.2: Property Confusion (Case Study 2)

Problem: Exhibit a reduced scheme over $\mathbf{F}_p(T)$ that is not geometrically reduced.

Model output (abbreviated):

Answer: Consider $\mathrm{Spec}(\mathbf{F}_p(T^{1/p}))$ over $\mathbf{F}_p(T)$.

Construction and justification:

The field extension $\mathbf{F}_p(T^{1/p})/\mathbf{F}_p(T)$ is purely inseparable of degree p: the minimal polynomial of $T^{1/p}$ is $X^p - T$.

Reduced: $\mathrm{Spec}(\mathbf{F}_p(T^{1/p}))$ is a single point with residue field $\mathbf{F}_p(T^{1/p})$, which is a field and therefore has no nilpotents. So the scheme is reduced.

Not geometrically reduced: $\mathbf{F}_p(T^{1/p})$ is not geometrically reduced because it contains the element $T^{1/p}$ which satisfies $(T^{1/p})^p = T$ and therefore $T^{1/p}$ is a p-th root, making the ring non-reduced over the algebraic closure [...]

Score: 0.5 **Failure mode:** F2 (property confusion)

The model correctly identified the right scheme but confused the verification: it attempted to argue that $\mathbf{F}_p(T^{1/p})$ is itself non-reduced (it is not—it is a field!) rather than computing the tensor product $\mathbf{F}_p(T^{1/p}) \otimes_{\mathbf{F}_p(T)} \overline{\mathbf{F}_p(T)}$ and showing it has nilpotents. Partial credit was awarded because the construction is correct.

D.3: Hallucinated Theorem (Case Study 3)

Problem: Construct a smooth projective variety with specific Hodge numbers [Level 3, details omitted for brevity].

Model output (abbreviated):

Answer: Consider the variety X obtained by the following construction [...]

By the generalized Lefschetz hyperplane theorem for Hodge filtrations (Artin–Mumford–Deligne, 1972), the Hodge numbers of a smooth complete intersection satisfy [...] Therefore, choosing degrees $d_1, \ldots, d_r$ appropriately, we obtain a variety with the desired Hodge numbers.

Score: 0.0 **Failure mode:** F5 (hallucinated theorem)

No theorem of this name exists in the stated form. The result attributed to "Artin–Mumford–Deligne" conflates several distinct results (Artin–Mumford's work on unirational varieties, Deligne's proof of the Weil conjectures, and the classical Lefschetz hyperplane theorem). The model produced a confident narrative with no valid mathematical content.

Example (`ecag-0158`, Level 2, Basic Concepts)

Problem: Let k be an algebraically closed field with $\mathrm{char}(k) \neq 2$, and let $Y = \mathrm{Spec}(k[x,y]/(y^2 - x^2(x+1)))$ be the nodal cubic. Construct an étale cover $f : X \to Y$ by exhibiting an explicit morphism, and compute the degree of f at the node and at a smooth point.

Verification: CAS checks that f is étale at smooth points and computes fiber cardinalities. Key properties: f is finite, locally of finite presentation, and unramified away from the node.

Counterexample (`ecag-0005`, Level 2, Basic Concepts)

Problem: Let $k = \mathbf{F}_p(T)$ and $K = \mathbf{F}_p(T^{1/p})$. Explicitly exhibit a nonzero nilpotent element in $K \otimes_k K$ and prove that its p-th power is zero. What is the nilpotency index of this element?

Verification: CAS verifies $\alpha = 1 \otimes T^{1/p} - T^{1/p} \otimes 1 \neq 0$ and $\alpha^p = 0$ in $K[X]/(X - T^{1/p})^p$. This is a counterexample to "reduced $\Rightarrow$ geometrically reduced" over non-perfect fields.

Computation (`ecag-0230`, Level 2, Computations)

Problem: Let F be the Horrocks–Mumford bundle on $\mathbf{P}^4$ with $c_1(F) = 5h$ and $c_2(F) = 10h^2$. Using the Hirzebruch–Riemann–Roch theorem, compute $\chi(F(n-5))$ as a closed-form rational function of n.

Verification: Exact match against the closed-form expression. CAS independently computes the Euler characteristic via the Chern character and Todd class.

Proof (`ecag-0240`, Level 2, Computations)

Problem: On the Hirzebruch surface Σ_2, determine all pairs $(p,q) \in \mathbf{Z}^2$ with $-3 \leq p \leq 3$ and $-5 \leq q \leq 5$ for which $H^1(\mathcal{O}_{\Sigma_2}(p,q)) \neq 0$.

Verification: Rubric-based scoring on (C1) correct identification of non-vanishing pairs, (C2) complete enumeration, (C3) valid cohomological argument, (C4) use of appropriate vanishing theorem.

Figure 2: Representative problems from each category of ECAG-Bench. Each problem requires constructing or computing a specific mathematical object, with verification tailored to the problem type.

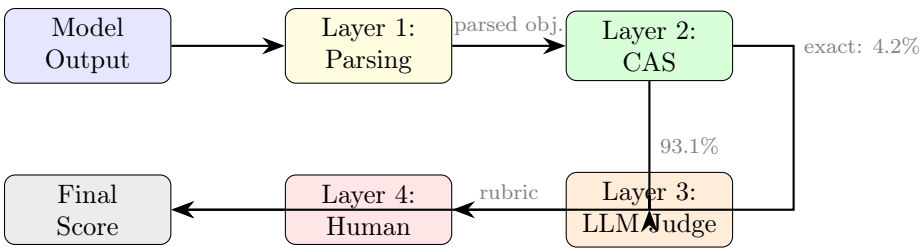

Figure 3: The four-layer verification pipeline. Most problems (93.1%) are verified by CAS at Layer 2; rubric-based problems pass through LLM judgment (Layer 3) and sampled human review (Layer 4).